Predictive Friend Recommendation System

DSA + STL + OOP | C++ Console Application

Team Division & Detailed Responsibility Guide

■ Task Distribution Note

Hussain and Talha have been assigned the two most difficult modules — the Graph+BFS Engine and the User+FileManager+RecommendationEngine. Ali and Haseeb handle the integration and testing modules which are equally important but less algorithmically complex.

1 **Hussain** — Graph Module + BFS Engine + Recommendation Engine — HARDEST

2 **Talha** — User Management + File Handling + Score Data — HARD

3 **Ali** — System Integration & Console UI — MODERATE

4 **Haseeb** — Testing, Utilities & Sample Data — MODERATE

What Is This Project?

This is a **Friend Recommendation System** — a C++ console program that works like a simplified version of the 'People You May Know' feature on Facebook or LinkedIn. The system reads a list of users and their existing friendships from text files, then predicts new friendships by calculating a **compatibility score** for each possible pair based on how many friends they share, how many interests they have in common, and how closely connected they are in the social graph.

Member	Difficulty	Owns	Key Idea	Files
Hussain	★★★ Hardest	Graph + RecommendationEngine	Build the graph & score predictions	Graph.h/.cpp RecommendationEngine.h/.cpp
Talha	★★★ Hard	User + FileManager	Store users & read/write files	User.h/.cpp FileManager.h/.cpp

Ali	★★ Moderate	FriendRecommendationSystem	Connect everything + menu	System.h/.cpp + main.cpp
Haseeb	★★ Moderate	Utilities + Test data	Helper functions + autoTest()	Helpers in System

Member 1 — Hussain | Graph + BFS + Recommendation Engine

HARDEST MODULE — You own two full classes and the core algorithm of the project

The Big Picture — What Are You Building?

Imagine a map of people. Every person is a dot (called a **node**), and every friendship is a line connecting two dots (called an **edge**). Your job is to build the code that stores this map, lets the program navigate it, and then uses it to predict who should become friends with whom.

■ *Think of it like a city road map. Nodes are cities, edges are roads. Your Graph class is the map itself. Your BFS function is the GPS that finds which cities you can reach within 2 road-hops. Your RecommendationEngine is the travel advisor that ranks those cities by how likely you are to want to visit them.*

PART A — The Graph Class

The Graph class stores all users and all friendships in memory. It uses a data structure called an **adjacency list**, which is just a dictionary where each user's ID maps to a list of their friends' IDs. It also provides functions to add/remove friendships and to search the graph.

Why an adjacency list and not a 2D array?

A 2D array (adjacency matrix) would need an N×N grid — if you have 1000 users, that's 1,000,000 cells, most of them empty. An adjacency list only stores actual friendships, so it uses far less memory and is much faster to traverse.

Functions You Need to Write in Graph

Function	What It Does (Plain English)	Input → Output
`addUser(id)`	Adds a new empty entry for this user in the adjacency list.	int id → void
`addFriend(u, v)`	Creates a friendship between u and v. Since friendships are two-way, you add v to u's list AND u to v's list.	int u, v → void
`removeFriend(u, v)`	Deletes the friendship. Remove v from u's list and u from v's list.	int u, v → void
`areFriends(u, v)`	Checks whether v is in u's friend list. Returns true or false.	int u, v → bool
`getFriends(id)`	Returns the full list of friend IDs for this user.	int id → vector<int>
`bfs(startId,maxDepth)`	Starting from one user, explore outward hop-by-hop. Return every user reachable within maxDepth steps, along with how many hops away they are.	int,int → map<id,distance>
`countMutualFriends(u,v)`	Count how many friends u and v share. This is used later in the scoring formula.	int u, v → int

`getAdjList()`	Returns a reference to the whole adjacency list so FileManager can save it to disk.	→ map<int,vector<int>>&

Understanding BFS — Step by Step

BFS stands for Breadth-First Search. It is the algorithm that discovers 'friends of friends.' Here is how it works in plain English, before you write a single line of code:

1	**Start at the target user.** Mark them as visited. Put them in a queue with depth = 0.
2	**Take the first person from the queue.** Look at all their friends (their neighbours in the adjacency list).
3	**For each friend not yet visited,** mark them visited, record their distance (current depth + 1), and add them to the queue.
4	**Repeat until the queue is empty** OR until you reach the maximum depth you care about (usually depth 2 or 3).
5	**Return the result map** which contains userId → distance for every reachable user. This is what the Recommendation Engine reads.

■ *Imagine you are standing in a room full of people and you shout your name. Everyone who hears you (your direct friends, depth 1) turns and shouts too. Everyone who hears THEM (friends of friends, depth 2) shouts next. BFS processes people in exactly this expanding-wave order, using a queue to track who shouts next.*

```
bfs(startId, maxDepth):
visited = unordered_set { startId }
q = queue of (userId, depth) pairs // start: (startId, 0)
result = empty map
while q is not empty:
(current, depth) = q.front(); q.pop()
if depth >= maxDepth: skip (don't go deeper)
for each neighbour in adjList[current]:
if neighbour not yet visited:
visited.add(neighbour)
result[neighbour] = depth + 1
q.push( (neighbour, depth + 1) )
return result
```

PART B — The RecommendationEngine Class

Once the Graph can find 'friends of friends,' the Recommendation Engine scores each of those candidates and ranks them. The score is a number that represents how likely two people are to become friends. A higher score means a stronger recommendation.

■ *Think of it like a matchmaking service. The Graph gives you a list of people the user doesn't know yet. The RecommendationEngine interviews each one and gives them a score out of 100 based on shared friends, shared hobbies, and how close they are. The top scorers get recommended.*

The Scoring Formula — How It Works

The score is made of four parts. Each part rewards something that suggests two people would get along:

```
Score = (mutualFriends x 10)
+ (sharedInterests x 7)
+ (interestSimilarity x 20) // value between 0.0 and 1.0
+ distanceBonus

distanceBonus = 15 if the candidate is 2 hops away
distanceBonus = 8 if the candidate is 3 hops away
distanceBonus = 0 otherwise
```

Worked Example:

```
User Ali vs User Talha:
3 mutual friends --> 3 x 10 = 30
2 shared interests --> 2 x 7 = 14
40% interest overlap --> 0.40 x 20 = 8
distance = 2 --> distanceBonus = 15
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Final Score = 30 + 14 + 8 + 15 = 67
```

What Is interestSimilarity and How Do You Calculate It?

Shared interests alone can be misleading. If Ali has 2 interests and Talha has 2, sharing 2 means 100% overlap. But if both had 10 interests and share 2, that's only 20%. The **Jaccard Similarity** formula normalises this:

```
interestSimilarity = sharedCount / (sizeOf_A + sizeOf_B - sharedCount)
Example: Ali = {coding, gaming} Talha = {coding, AI, gaming}
shared = 2 (coding, gaming)
union = 3 (coding, gaming, AI)
Jaccard = 2 / 3 = 0.67 (67%)
```

How to Implement countSharedInterests():

Convert both users' interest vectors into unordered_sets, then loop through the smaller set and check if each interest exists in the other set. Each hit is a shared interest.

How generateRecommendations() Should Work — Full Pipeline

1. **Call bfs(targetId, 3)** Get a map of every user within 3 hops and their distances. These are your candidates.
2. **Build an exclusion set** Put the target user's existing friends (and themselves) into an unordered_set<int>. You will skip these.

3	**Loop through every candidate from BFS** If the candidate is in the exclusion set, skip them. Otherwise proceed.
4	**Call calculateScore() for this candidate** This calls countMutualFriends() from Graph, countSharedInterests() and calculateInterestSimilarity() from your own engine, and reads the distance from the BFS result.
5	**Store as a Recommendation struct** Fill in all fields: userId, mutualFriends, sharedInterests, interestSimilarity, distance, score.
6	**Call rankRecommendations()** Use std::sort with a custom comparator to sort descending by score.
7	**Return the first topK entries** This is the final ranked list that Ali will display in the menu.

STL Containers You Use and Why

STL Container / Concept	What It Is (Plain English)	Why YOU Need It
`unordered_map<int, vector<int>>`	A dictionary: each user ID maps to a list of their friends' IDs	This IS the graph. Every graph operation starts here.
`queue<pair<int,int>>`	A first-in-first-out line (like a queue at a counter)	BFS needs to process nodes in order of discovery. A queue does exactly this.
`unordered_set<int>`	A set of integers with O(1) membership check	BFS uses this as a 'visited' tracker so it never revisits a node.
`vector<Recommendation>`	A resizable list of Recommendation structs	Holds all scored candidates before sorting.
`unordered_set<string>`	A set of strings with O(1) lookup	Used to count shared interests — convert both interest lists to sets, then check intersections.
`std::sort + lambda comparator`	A built-in sorting function that you customise	Sorts the Recommendation vector descending by score so the best candidate is first.

■ *Build and test the Graph class completely before starting RecommendationEngine. Print the BFS result for User 1 with depth=2 and verify it manually on paper first.*

■ *In your viva, say 'Jaccard similarity' when explaining interestSimilarity. It is the formal name for this formula and it will impress your instructor.*

■ *Your generateRecommendations() can directly call this->bfs() because RecommendationEngine receives a Graph reference. Call graph.bfs(targetId, 3) inside the function.*

Member 2 — Talha | User Class + File Handling

HARD MODULE — You are the data foundation. Nothing works without your code.

The Big Picture — What Are You Building?

Every program needs to store its data somewhere. Your job is to define **what a User looks like** in code, and to write the functions that **load user data from text files** on startup and **save it back** when the program exits. You are building the data layer — the foundation that every other class in the project depends on.

■ *Think of yourself as the librarian. Hussain, Ali, and Haseeb are readers who need books (User objects). You are responsible for: (1) deciding what information goes on each book's cover (the User class fields), and (2) fetching books from the shelf (loading from files) and returning them to the shelf when done (saving to files).*

PART A — The User Class

A User object represents one person in the system. It stores their ID number, their name, and a list of their interests. You also need to provide two helper methods that the rest of the project calls.

Fields Inside the User Class

Field	Type	What It Stores	Example Value
id	int	A unique number identifying this user	1
name	string	The user's full name	Ali
interests	vector<string>	A list of the user's hobbies/interests	{coding, gaming, cricket}

Methods Inside the User Class

Function	What It Does (Plain English)	Input → Output
`User()`	Default constructor. Creates an empty User object. Required so you can store Users in containers.	no input → empty User
`User(id, name, interests)`	Parameterised constructor. Takes all three fields and stores them. This is the constructor you use when loading from files.	int, string, vector → User
`interestSet()`	Converts the interests vector into an unordered_set<string>. Why? Because Hussain's engine needs to check 'does this user like cricket?' very fast (O(1)). Checking a vector is slow (O(n)); checking a set is instant.	→ unordered_set<string>
`display()`	Prints the user's details neatly to the console. Used by Ali's menu when the user asks to view a profile.	→ void (prints to console)

PART B — The FileManager Class

The FileManager class handles everything to do with reading from and writing to text files. On startup, Talha's FileManager reads two files — users.txt and friends.txt — and converts them into C++ data structures that the rest of the program uses. On exit, it writes everything back so changes are not lost.

The File Formats You Must Support

```
FILE: users.txt
Format: id,name,interest1,interest2,interest3
Example lines:
1,Ali,coding,gaming,cricket
2,Hussain,coding,cycling,AI
3,Haseeb,cricket,movies,gaming

FILE: friends.txt
Format: userId1 userId2 (means these two are friends)
Example lines:
1 2 <- Ali and Hussain are friends
1 3 <- Ali and Haseeb are friends
2 4 <- Hussain and Talha are friends
```

Functions in FileManager — What Each One Does

Function	What It Does (Plain English)	Input → Output
`loadUsers(filename)`	Opens users.txt. Reads it line by line. For each line, splits it by commas to get the id, name, and interests. Creates a User object and puts it into a map. Returns the complete map when done.	string filename → unordered_map<int,User>
`loadFriendships(filename)`	Opens friends.txt. Reads each line as two numbers separated by a space. Returns a list of (id1, id2) pairs. Ali's System will pass these pairs to Hussain's graph.addFriend() one by one.	string filename → vector<pair<int,int>>
`saveUsers(users, filename)`	Takes the users map and writes each User back to users.txt in the same comma format. Called when the program exits or when the user chooses 'Save' from the menu.	map + string → void (writes file)
`saveFriendships(adjList, filename)`	Takes Hussain's adjacency list and writes each friendship to friends.txt. Be careful not to write each pair twice (since the graph stores u→v AND v→u).	adj list + string → void (writes file)
`parseInterests(line)`	Takes a string like 'coding,gaming,cricket' and splits it into a vector: {'coding','gaming','cricket'}. This is called by loadUsers() for every line in the file.	string → vector<string>

How to Split a Comma-Separated String (parseInterests)

This is the trickiest part of your code. Here is the exact approach to use:

```
// Input: line = '1,Ali,coding,gaming,cricket'
```

```
// Step 1: wrap it in a stringstream
stringstream ss(line);
string token;
vector parts;
// Step 2: split by comma, collect each piece
while (getline(ss, token, ',')) {
parts.push_back(token);
}
// Result: parts = {'1', 'Ali', 'coding', 'gaming', 'cricket'}
// parts[0] = id, parts[1] = name, parts[2..] = interests
```

STL Containers You Use and Why

STL Container / Concept	What It Is (Plain English)	Why YOU Need It
`vector<string> interests`	A resizable list of strings	Stores all the interests for one user. Easy to iterate and pass to other functions.
`unordered_map<int, User>`	A dictionary mapping int keys to User values	loadUsers() builds this map. The whole program uses it to look up any user by ID in O(1) time.
`unordered_set<string>`	A set of strings with instant membership check	interestSet() returns this. Hussain's engine calls it to check shared interests in O(1).
`ifstream`	Input file stream — reads a file	Used in loadUsers() and loadFriendships() to open and read text files line by line.
`ofstream`	Output file stream — writes a file	Used in saveUsers() and saveFriendships() to write data back to disk.
`stringstream`	Treats a string as a stream so you can read from it	Used inside parseInterests() to split a comma-separated line into tokens.

■ *Write and test parseInterests() first in isolation. Call it with 'coding,gaming,AI' and print each token. Once that works, loadUsers() becomes straightforward.*

■ *In saveFriendships(), loop through the adjacency list but only write an edge u v when u < v. This prevents writing '1 2' and '2 1' as duplicate entries.*

■ *Create users.txt and friends.txt with at least 8 users on Day 1 and share with the group. Hussain needs them to test BFS. Ali needs them to test the menu.*

Member 3 — Ali | System Integration & Console UI

MODERATE MODULE — You connect all the pieces and build the user-facing menu.

The Big Picture — What Are You Building?

You are the **integrator**. Hussain builds the Graph and the engine. Talha builds the data layer. You write the code that **connects these modules together** and presents them to the user through a console menu. You also write **main.cpp**, which is the entry point — the first file that runs when the program starts.

■ *Think of yourself as the manager of a restaurant. Hussain is the chef (he cooks the recommendations). Talha runs the pantry (he stores and fetches ingredients). Haseeb is the quality inspector. You are the front-of-house manager — you take the customer's order (menu input), send it to the right kitchen station, and bring the result back to the customer (display output).*

Your Class: FriendRecommendationSystem

This class lives in system.h and system.cpp. It holds the main users map (unordered_map<int, User>), an instance of Hussain's Graph, and an instance of Hussain's RecommendationEngine. Every menu action calls methods on these objects.

Functions You Need to Write

Function	What It Does (Plain English)	Input → Output
`run()`	The main program loop. Shows the menu, reads the user's choice, calls the right function via a switch statement, repeats until the user enters 0 to exit.	→ void (runs forever until exit)
`showMenu()`	Prints the numbered list of options to the console. Call this at the top of every loop iteration.	→ void (prints to console)
`registerUser()`	Asks the user to type a name and interests. Auto-assigns the next available ID (e.g. if 15 users exist, new user gets ID 16). Adds the new User to the map AND calls graph.addUser(id).	→ void (modifies users map and graph)
`addFriendMenu()`	Asks for two user IDs. Validates that both exist and are not already friends. Calls graph.addFriend(u, v).	→ void (modifies graph)
`removeFriendMenu()`	Asks for two user IDs. Validates both exist and are currently friends. Calls graph.removeFriend(u, v).	→ void (modifies graph)
`viewFriends(id)`	Gets the friend list from graph.getFriends(id). For each friend ID, looks up the User in the map and prints their name.	int id → void (prints list)

`viewMutualFriends(u,v)`	Gets both users' friend lists. Loops through one list and checks which IDs appear in the other. Prints the matching names.	int u, v → void (prints list)
`displayRecommendations(id)`	Calls engine.generateRecommendations(id, graph, users, 5). Calls Haseeb's printRecommendationTable() to display the ranked results.	int id → void (prints table)
`searchUser(query)`	Converts the query to lowercase. Loops through all users, converts each name to lowercase, checks if it contains the query as a substring. Prints all matches.	string query → void (prints matches)
`listAllUsers()`	Loops through the users map and prints each user's ID and name in a formatted list.	→ void (prints list)
`loadSampleData()`	Calls FileManager.loadUsers() and FileManager.loadFriendships(). For each friendship pair returned, calls graph.addUser() and graph.addFriend() to build the graph.	→ void (populates map and graph)
`saveData()`	Calls FileManager.saveUsers() and FileManager.saveFriendships(graph.getAdjList()) to write current state to disk.	→ void (writes files)

The Menu — What It Looks Like

```
===== Friend Recommendation System =====
1. Load data from files
2. Register new user
3. Add friendship
4. Remove friendship
5. View all users
6. View a user's friends
7. View mutual friends
8. Get friend recommendations
9. Search user by name
10. Save data to files
0. Exit
========================================
Enter your choice: _
```

Input Validation — Every Menu Action Needs This

Bad input will crash the program if you don't handle it. For every menu action that takes a user ID, check these conditions before proceeding:

Situation	Check	Error Message to Show
User enters a non-integer choice	cin.fail() after reading	Invalid input. Please enter a number.
User enters an ID that does not exist	users.count(id) == 0	No user found with that ID.

User tries to friend themselves	u == v	A user cannot be friends with themselves.
Users are already friends	graph.areFriends(u,v) == true	These users are already friends.
Users are not friends (for remove)	graph.areFriends(u,v) == false	These users are not currently friends.
Name field is left blank	name.empty() after getline	Name cannot be empty. Please try again.

How to Handle Bad Integer Input Safely

```
int choice;
cin >> choice;
if (cin.fail()) {
cin.clear(); // reset error state
cin.ignore(1000, '\n'); // discard the bad input from the buffer
cout << 'Invalid input.' << endl;
continue; // go back to top of menu loop
}
```

STL / Concepts You Use and Why

STL Container / Concept	What It Is (Plain English)	Why YOU Need It
`unordered_map<int, User>`	The main user store (dictionary)	Passed by reference to Graph and RecommendationEngine. Look up any user by ID instantly.
`for (auto& [id, user] : users)`	Range-for loop over all map entries	Use this in listAllUsers() and searchUser() to iterate every registered user.
`std::transform + ::tolower`	Converts a string to all-lowercase characters	Used in searchUser() so 'ali' matches 'Ali' regardless of how the user types it.
`string::find(query)`	Checks if a substring exists inside a string	Used in searchUser() to find partial name matches.
`cin.clear() + cin.ignore()`	Input stream error recovery	Clears the error flag and discards leftover input after a bad read.
`std::setw + std::left`	Output formatting from <iomanip>	Makes the recommendation table columns align neatly. Looks professional in a demo.

■ *Start by hardcoding 5 users directly in loadSampleData() so you can test the full menu before Talha's file loader is ready. Replace it with the real file loading later.*

■ *Use std::setw(15) and std::left to pad names to a fixed width in the recommendation table. It makes a huge visual difference in a viva demo.*

■ *Your loadSampleData() does TWO things: calls FileManager to get the users map, then loops through the friendships list and calls graph.addFriend() for each pair. Don't forget the second step or the graph will be empty.*

Member 4 — Haseeb | Testing, Utilities & Sample Data

MODERATE MODULE — You are the quality backbone. Your work makes everyone else's look good.

The Big Picture — What Are You Building?

Your role has three responsibilities. First, you create the **sample dataset** (the text files) that all four members use to test their code from Day 1. Second, you write a set of **utility helper functions** that Ali calls from the System class — these handle display formatting, input safety, and validation. Third, you write an **autoTest() function** that runs a sequence of automated checks and prints PASS or FAIL for each one.

■ *Think of yourself as the quality inspector on an assembly line. The other three members build the parts. You prepare the test materials (sample data), provide the measurement tools (utility functions), and run the final inspection (autoTest). Without your test data, no one can verify their code works.*

PART A — Prepare the Sample Dataset (Do This First)

Create these two text files and share them with the group immediately. Every member needs them to test their code. Do not wait for other parts of your work to be complete before sharing.

users.txt (15 users minimum)

```
1,Ali,coding,gaming,cricket
2,Hussain,coding,cycling,AI
3,Haseeb,cricket,movies,gaming
4,Talha,AI,coding,reading
5,Sara,movies,music,painting
6,Zain,gaming,coding,cricket
7,Maryam,reading,AI,cooking
8,Bilal,cycling,cricket,photography
9,Nadia,music,painting,dancing
10,Usman,coding,reading,chess
11,Fatima,cooking,movies,music
12,Kamran,AI,chess,cycling
13,Sana,photography,painting,music
14,Asad,cricket,gaming,cycling
15,Hira,reading,cooking,AI
```

friends.txt (30 connections minimum — a sample below)

```
1 2 1 3 1 6 2 4 2 7 3 8 3 14
4 7 4 10 5 9 5 11 6 14 7 12 7 15
8 12 8 14 9 11 9 13 10 12 10 15 11 13
... add more pairs until you have at least 30 total friendships
```

Design the friendships so that User 1 (Ali) has at least 3 friends, and those friends also have friends among themselves. This creates a realistic 'friends of friends' network that will produce visible recommendations when Hussain tests his BFS.

PART B — Utility Helper Functions

These functions are written inside (or alongside) Ali's System class. You write them, Ali calls them. Each one is a single focused task — short to write but important for making the output look professional.

Function	What It Does (Plain English)	Input → Output
`printDivider(char c, int w)`	Prints a horizontal line of character c, w characters wide. Used between sections in the menu.	char, int → void (prints line)
`printUserCard(User& u)`	Prints one user's profile in a neat formatted box showing their ID, name, and interest list. Ali calls this in viewFriends() and searchUser().	User ref → void (prints box)
`printRecommendationTable(vector<Recommendation>& recs)`	Prints the ranked recommendation list in a table with columns for rank, name, score, mutual friends, shared interests, and distance. Ali calls this in displayRecommendations().	vector ref → void (prints table)
`validateUserExists(int id, unordered_map<int,User>& users)`	Checks if the given id key exists in the users map. Returns true if found, false otherwise. Ali calls this before every menu action that takes a user ID.	int, map ref → bool
`toLowerString(string s)`	Returns a lowercase copy of the given string. Used inside Ali's searchUser() so the search is case-insensitive.	string → string
`getIntInput(string prompt)`	Prints the prompt, reads an integer from cin, handles invalid (non-integer) input by clearing the buffer and asking again, and returns the valid integer.	string prompt → int
`autoTest(Graph& g, unordered_map<int,User>& users, RecommendationEngine& engine)`	Runs a series of automated assertions on the loaded data. Prints PASS or FAIL for each test. Call this after data is loaded.	refs → void (prints results)

What printRecommendationTable() Should Look Like

```
=== Top Friend Recommendations for Ali (ID: 1) ===
Rank Name Score Mutuals Shared Distance
---- --------------- ------ ------- ------ --------
1 Talha (ID: 4) 67.0 3 2 2
2 Sara (ID: 7) 52.0 2 3 2
3 Zain (ID: 9) 34.0 1 1 3
// Use std::setw() and std::left from to align columns
```

PART C — The autoTest() Function

autoTest() runs after the data is loaded and checks that everything is working correctly. You call it from main.cpp just before handing control to the menu. For each test, print a clear PASS or FAIL message.

1 **Test 1 — Data loaded correctly** Check users.size() == 15. If yes → PASS. If no → FAIL.

2	**Test 2 — Graph is populated** Call graph.getFriends(1). Check it is not empty. PASS / FAIL.
3	**Test 3 — BFS works** Call graph.bfs(1, 2). Check the result is not empty. PASS / FAIL.
4	**Test 4 — Mutual friend count is non-negative** Call graph.countMutualFriends(1, 6). Check result >= 0. PASS / FAIL.
5	**Test 5 — Recommendations are generated** Call engine.generateRecommendations(1, graph, users, 5). Check result.size() > 0. PASS / FAIL.
6	**Test 6 — Scores are positive** Check recs[0].score > 0. PASS / FAIL.
7	**Test 7 — Existing friends not in recommendations** Get Ali's friends list. Check none of those IDs appear in the recommendations. PASS / FAIL.

STL / Concepts You Use and Why

STL Container / Concept	What It Is (Plain English)	Why YOU Need It
`std::setw(n) + std::left`	Fixed-width column formatting from <iomanip>	Makes printRecommendationTable() look like a real table with aligned columns.
`string::size()`	Returns the length of a string	Used in printDivider() to match the width of a header line automatically.
`std::transform + ::tolower`	Converts every character in a string to lowercase	Used in toLowerString() which Ali's searchUser() calls.
`cin.fail()`	Returns true if the last cin read failed (e.g. got text instead of int)	Used in getIntInput() to detect and recover from bad user input.
`assert / if-checks`	Conditional checks used for testing	Used in autoTest() to verify that loaded data and function outputs are correct.

■ *Do PART A first — create and share the data files before writing any code. The whole team is blocked without test data.*

■ *For printRecommendationTable(), include all 6 columns: rank, name, score, mutualFriends, sharedInterests, distance. This shows your instructor that the scoring formula is working correctly.*

■ *Run autoTest() just before the viva. If any test fails, tell the owning member immediately — Hussain fixes Graph/Engine bugs, Talha fixes data bugs, Ali fixes menu bugs.*

Integration Plan & Viva Preparation

Build Order — Follow This Sequence

Phase 1	**Haseeb**	Create users.txt (15 users) and friends.txt (30 connections). Share with the group immediately. Everyone needs these files to test their code.
Phase 2	**Talha**	Build User.h/.cpp. Test the constructor and interestSet(). Then build FileManager. Test loadUsers() by printing all loaded users to console.
Phase 3	**Hussain**	Build Graph.h/.cpp. Test addFriend and getFriends. Then test bfs() manually. Build RecommendationEngine. Test calculateScore() with known expected values.
Phase 4	**Ali**	Build FriendRecommendationSystem. Hardcode 5 users first, test the menu fully. Then replace with real FileManager calls and re-test.
Phase 5	**Haseeb**	Add utility helpers to System. Run autoTest(). Report any bugs to the member who owns that file.
Phase 6	**All**	Final link. Haseeb runs: g++ -std=c++17 main.cpp user.cpp graph.cpp recommendation.cpp system.cpp -o frs. Fix any linking errors together.

Compile Commands

Member	Command
Talha	g++ -std=c++17 -c user.cpp -o user.o
Hussain	g++ -std=c++17 -c graph.cpp -o graph.o
Hussain	g++ -std=c++17 -c recommendation.cpp -o recommendation.o
Ali	g++ -std=c++17 -c system.cpp -o system.o
Ali (final link)	g++ -std=c++17 main.cpp user.cpp graph.cpp recommendation.cpp system.cpp -o frs

Viva Questions — What Your Teacher Will Ask

Q: What is a graph in the context of this project?

A graph is a data structure made of nodes (users) and edges (friendships). In our project, each user is a node, and every friendship is an undirected edge connecting two nodes. We store this using an adjacency list — a map where each user ID points to a list of their friends' IDs.

Q: Why did you use BFS instead of DFS?

BFS (Breadth-First Search) explores all nodes at the current distance before going further. This means it finds all friends-of-friends (depth 2) before going to depth 3. DFS would go deep into one branch first and could return candidates that are much further away. BFS is correct for finding the nearest candidates for recommendation.

Q: Explain your recommendation score formula.

The score has four components: mutual friends multiplied by 10 (the strongest signal), shared interests multiplied by 7, interest similarity (Jaccard index) multiplied by 20, and a distance bonus of 15 for depth-2 candidates or 8 for depth-3. Higher weights go to signals that are stronger predictors of likely friendship.

Q: What is the Jaccard similarity index?

Jaccard similarity measures the overlap between two sets as: shared / (|A| + |B| - shared). It normalises the shared interest count relative to the total number of unique interests between the two users. This prevents users with many interests from being unfairly rewarded just for having a large interest list.

Q: Why use unordered_map instead of map?

unordered_map uses hashing to achieve O(1) average lookup, insert, and delete. map uses a balanced BST which gives O(log n) for the same operations. Since our program looks up users by ID very frequently, O(1) is significantly faster.

Q: What OOP principles does your project demonstrate?

Encapsulation: each class hides its data (e.g. Graph's adjList is private). Separation of concerns: User stores data, FileManager handles files, Graph manages structure, Engine computes scores, System handles UI. Composition: System contains instances of all other classes.